REPORTE UNIFICADO DE VULNERABILIDADES

Módulo: actividades_complementarias — Odoo 19 Community

Tipo: Pruebas Unitarias + Verificación Red Team Manual + Hallazgos de Sistema

ISO 29119 · OWASP Top 10 · Abril 2026

Fecha	Abril 2026
Módulo	actividades_complementarias
Tests ejecutados	290
Fallidos documentados	11 (vulnerabilidades documentadas)
Errores técnicos	0
Secciones	I — Pruebas Unitarias (RN) II — Red Team Manual III — Hallazgos de Sistema (SYS)

Índice General de Hallazgos

#	ID	Título	Severidad
SECCIÓN I — Pruebas Unitarias (Reglas de Negocio)			
01	**RN-11**	bypass_edit_protection accesible sin privilegios	**CRÍTICO**
02	**RN-15**	responsable_bloqueado no se valida en write()	**ALTO**
03	**RN-05**	Actividad no predefinida al catálogo sin aprobación del Comité	**ALTO**
04	**RN-10**	action_firmar_constancias() no valida la identidad del firmante	**ALTO**
05	**RN-07**	skip_fecha_check accesible externamente en create()	**ALTO**
06	**RN-04**	JD puede manipular fecha_limite_revision de su propuesta	**ALTO**
07	**RN-01**	JD-B puede acceder a registros de actividades del JD-A	**ALTO**
08	**RN-08**	Horas cero no rechazadas cuando skip_horas_check está activo	**MEDIO**
09	**TEST-01**	assertIn('Cupo') falla por diferencia de mayúsculas	**AJUSTE**
SECCIÓN II — Verificación Red Team Manual			
H1	**RED-01**	Omisión Lógica en Límite de Horas Diarias (Zero-Day)	**CERRADO**

H2	RED-02	Evasión del Comité Académico mediante Inyección de ORM	CRÍTICO
H3	RED-03	Validación de Créditos Ilegales (ORM Protegido)	CERRADO
SECCIÓN III — Hallazgos de Sistema (SYS)			
10	RN-16	Validación estándar omite restricción de horas en cero	MEDIO
11	SYS-01	Parámetro 'options' inválido en modelo Python	BAJO
12	SYS-02	Sintaxis obsoleta '_sql_constraints' en modelos	BAJO

SECCIÓN I — Pruebas Unitarias

Módulo: actividades_complementarias · Odoo 19 Community · ISO 29119 · OWASP Top 10

Tests ejecutados	290
Fallidos	11 (intencionales — vulnerabilidades documentadas)
Errores técnicos	0

01 [RN-11] bypass_edit_protection accesible sin privilegios — CRÍTICO

Módulo afectado	actividad.complementaria
Archivo / Método	models/actividad.py — método write()

Tests que detectaron este problema

- test_bypass_context_no_funciona_para_jd_sin_sudo
- test_rn11_bypass_context_no_accesible_sin_sudo
- test_rn11_comite_no_puede_usar_bypass

¿Qué hizo la prueba unitaria?

La prueba unitaria creó una actividad complementaria en estado en_revision asignada al JD-A. Luego intentó ejecutar un write() sobre esa actividad usando el usuario JD-A (sin permisos de administrador) pero pasando el context flag bypass_edit_protection=True. La expectativa era que el sistema rechazara la operación con un UserError, pero el sistema la ejecutó con éxito, modificando el estado del registro directamente.

Impacto en el sistema

La bandera bypass_edit_protection fue diseñada exclusivamente para uso interno del sistema (cron jobs, wizards, acciones de negocio que requieren modificar campos auto-gestionados). El método write() en actividad.py verifica este flag al inicio y, si está presente, omite completamente todas las validaciones de seguridad, restricciones de estado y protecciones de campos. Cualquier usuario autenticado con acceso a la API JSON-RPC de Odoo puede pasar este contexto en una llamada directa al servidor y saltarse la totalidad de las reglas de negocio implementadas.

Resultado de la verificación manual

Falso Positivo. (El backend bloqueó la inyección)

Código actual (problemático)

```
# actividad.py — write() actual (VULNERABLE)
def write(self, vals):
    if self.env.context.get('bypass_edit_protection'):
        return super().write(vals)  # <-- omite TODAS las validaciones
    ...
```

Solución propuesta

Restringir el bypass únicamente a contextos de sistema (env.su = True) o al grupo administrador. Los wizards y acciones de negocio internas ya usan sudo() antes de pasar el context, por lo que este cambio no rompe el flujo interno.

```
# actividad.py — write() corregido
def write(self, vals):
    if self.env.context.get('bypass_edit_protection'):
        is_admin = self.env.user.has_group(
            'actividades_complementarias.group_admin_actividades'
        )
        if not (self.env.su or is_admin):
            raise UserError(_('Operacion no permitida.'))
        return super().write(vals)
    ...
```

02 [RN-15] Falsificación de Firma Digital — ALTO

Módulo afectado	actividad.complementaria
Archivo / Método	models/actividad.py — método write()

Tests que detectaron este problema

- test_responsable_bloqueado_impide_sobrescritura
- test_rn15_responsable_bloqueado_impide_write_directo

¿Qué hizo la prueba unitaria?

Cualquier usuario puede firmar constancias académicas como si fuera el JD o el Responsable. Las constancias firmadas se convierten en documentos oficiales con validez legal.

Resultado de la verificación manual

1. Falsificación de Firma del JD: Parcialmente seguro. (Los agentes recomiendan encarecidamente validar la identidad en el backend y no solo confiar en la visibilidad del botón en la UI.)
2. Falsificación de Firma del Responsable: Falso Positivo. (El backend rechazó la falsificación.)

Impacto en el sistema

El campo responsable_bloqueado=True es el mecanismo de integridad que garantiza que, una vez confirmado formalmente el responsable de una actividad a través del wizard dedicado (con doble confirmación), ese dato no pueda modificarse silenciosamente. Si el JD puede sobreescribirlo directamente vía write(), puede cambiar el responsable de una actividad que ya está en curso o pendiente de inicio sin dejar rastro de la operación original.

Código actual (problemático)

```
def action_firmar_constancias(self):
    # Solo verifica estado y flag, NUNCA verifica quién firma
    self.with_context(bypass_edit_protection=True).write({'jd_firmo': True})
```

```
def action_firmar_constancias_responsable(self):
    # Mismo problema
    self.with_context(bypass_edit_protection=True).write({'responsable_firmo':
True})
```

Solución propuesta

```
def action_firmar_constancias(self):
    self.ensure_one()
    if self.estado_code != 'finalizada': raise ValidationError(...)
    if self.jd_firmo: raise ValidationError(...)
    # ✅ IDENTITY CHECK
    is_admin =
self.env.user.has_group('actividades_complementarias.group_admin_actividades')
    if not is_admin and self.jefe_departamento_id.id != self.env.user.id:
        raise UserError(_('Solo el Jefe de Departamento asignado (%s) puede
firmar.')
            % self.jefe_departamento_id.name)
    self._system_write({'jd_firmo': True})

def action_firmar_constancias_responsable(self):
    self.ensure_one()
    if self.estado_code != 'finalizada': raise ValidationError(...)
    if self.responsable_firmo: raise ValidationError(...)
    # ✅ IDENTITY CHECK
    if self.responsable_actividad_id.id != self.env.user.id:
        raise UserError(_('Solo el Responsable asignado (%s) puede firmar.')
            % self.responsable_actividad_id.name)
    self._system_write({'responsable_firmo': True})
```

03 [RN-05] SQL Crudo — Bypass de Record Rules — ALTO

Módulo afectado	actividad.complementaria
Archivo / Método	models/actividad.py — método compute_dominios()

Tests que detectaron este problema

- test_proxima_audit.py

¿Qué hizo la prueba unitaria?

El método _compute_dominios() ejecuta SQL directo sobre res_groups_users_rel, lo que bypasea todas las record rules del ORM.

Impacto en el sistema

Bypass de record rules del ORM, multi-company isolation, y field ACLs. En instancias multi-empresa, filtra IDs de usuarios de otras compañías sin respetar el aislamiento definido.

Código actual (problemático)

```
self.env.cr.execute(
    """SELECT uid FROM res_groups_users_rel WHERE gid = %s""",
    (grupo.id,)
)
```

Solución propuesta

Verificación de los Agentes: Confirmado como anti-patrón de seguridad en Odoo.

```
def _compute_dominios(self):
    def _user_ids_en_grupo(xmlid):
        grupo = self.env.ref(xmlid, raise_if_not_found=False)
        if not grupo:
            return []
        return grupo.sudo().user_ids.ids  # ✅ ORM respeta record rules
```

04 [RN-10] Escalada de Privilegios: Escritura Directa — ALTO

Módulo afectado	actividad.complementaria
Archivo / Método	models/actividad.py — método action_firmar_constancias()

Tests que detectaron este problema

- test_rn10_responsable_no_puede_firmar_constancias_jd
- test_rn10_jd_ajeno_no_puede_firmar_constancias

¿Qué hizo la prueba unitaria?

La prueba finalizó una actividad (estado finalizada), le asignó un responsable, y luego llamó action_firmar_constancias() usando el usuario Responsable de Actividad (no el JD). Se esperaba un UserError porque la regla de negocio establece que únicamente el Jefe de Departamento puede firmar constancias. Sin embargo, el método ejecutó la firma completa, marcó jd_firmo=True y generó el ZIP de constancias usando el nombre del Responsable como si fuera el JD.

Impacto en el sistema

Permitir a usuarios sin rol definido escribir campos protegidos como estado_id o flags de firma directamente, saltando el flujo de aprobación institucional.

Código actual (problemático)

```
if not is_jd:
    continue  # ← PASA SIN RESTRICCIÓN para alumnos, public, etc.
```

Solución propuesta

Implementar política de default_deny al final del método write(), permitiendo solo a env.su o roles explícitamente autorizados.

```
def write(self, vals):
    # ✅ Permitir al superusuario (crons, upgrades)
    if self.env.su:
        return super().write(vals)
    # ... validaciones de admin, JD, personal ...
    # ✅ DEFAULT DENY — ningún otro rol puede escribir
    raise UserError(_('No tiene permiso para modificar actividades
complementarias.'))
```

05 [RN-07] IDOR en Descarga de Constancias — CRÍTICO

Módulo afectado	actividad.complementaria
Archivo / Método	models/actividad.py — constraint _check_fechas()

Descripción

El attachment ZIP de constancias se crea sin access_token y la URL de descarga es predecible, lo que permite a un atacante enumerar IDs para descargar constancias de otros departamentos.

Impacto en el sistema

El attachment id=843 no tiene access_token. Un atacante puede probar id=842, 841, 840... para descargar constancias de otros departamentos sin autorización.

Código actual (problemático)

```
attachment = self.env['ir.attachment'].sudo().create({...})
# URL: /web/content/827?download=true  ← ID secuencial, sin token
```

Solución propuesta

Generar un access_token aleatorio al crear el attachment y usarlo en la URL de descarga.

```
attachment = self.env['ir.attachment'].sudo().create({...})
access_token = attachment.sudo()._generate_access_token()
return {
    'type': 'ir.actions.act_url',
    'url':
f'/web/content/{attachment.id}?access_token={access_token}&download=true',
    'target': 'self',
}
```

06 [RN-04] Escalada de Privilegios ORM — Auto-Aprobación por Jefe de Departamento

ALTO

Módulo afectado	actividad.propuesta
Archivo / Método	models/propuesta_actividad.py — campo fecha_limite_revision

Tests que detectaron este problema

- test_jd_no_puede_retroceder_fecha_limite_revision
- test_rn04_jd_no_puede_retroceder_fecha_limite

¿Qué hizo la prueba unitaria?

Automática: Revisar test_03_escalada_privilegios_jefe_departamento en tests/test_exploracion_propuesta.py. Manual (RPC): JD envía una propuesta a revisión. Usando un cliente XML-RPC, ejecuta un método write sobre actividad.propuesta apuntando al ID de su propuesta y pasando {'estado_solicitud_id': ID_ESTADO_APROBADA}. La propuesta quedará aprobada instantáneamente sin validación.

Impacto en el sistema

El Jefe de Departamento (JD) cuenta con permisos perm_write=1 sobre el modelo actividad.propuesta. Ya que el modelo en Python carece de un método write() sobreescrito para

restringir campos de solo lectura, el JD puede editar vía ORM el campo estado_solicitud_id. Esto permite que un JD se auto-apruebe su propia propuesta, bypasseando por completo al Comité Académico.

Código actual (problemático)

```
def write(self, vals):
    if not
self.env.user.has_group('actividades_complementarias.group_comite_academico') and
\
        not
self.env.user.has_group('actividades_complementarias.group_admin_actividades'):
         if 'estado_solicitud_id' in vals:
             raise AccessError('Solo el Comité Académico puede modificar el estado
de la propuesta.')
    return super().write(vals)
```

Solución propuesta

Hacer el campo fecha_limite_revision readonly a nivel ORM. Reducir los permisos del JD sobre el modelo propuesta a solo lectura.

```
def write(self, vals):
    if not
self.env.user.has_group('actividades_complementarias.group_comite_academico') and
\
        not
self.env.user.has_group('actividades_complementarias.group_admin_actividades'):
         if 'estado_solicitud_id' in vals:
             raise AccessError(
                 'Solo el Comité Académico puede modificar el estado de la
propuesta.'
             )
    return super().write(vals)
```

07 [RN-01] Falla en Máquina de Estados — CRÍTICO

Módulo afectado	actividad.complementaria
Archivo / Método	models/propuesta_actividad.py — action_aprobar() y action_rechazar()

Tests que detectaron este problema

- test_rn01_jd_b_no_puede_leer_actividad_de_jd_a_directamente

¿Qué hizo la prueba unitaria?

Automática: Ejecutar test_02_maquina_estados_aprobar_rechazada. El test invoca action_aprobar() después de action_rechazar(). La transición ocurre limpiamente, lo cual es ilegal en términos de lógica de negocio pura.

Impacto en el sistema

Los métodos action_aprobar y action_rechazar no validan que la propuesta se encuentre en el estado lógico inicial correcto (en_revision). Un actor con acceso al Comité puede invocar action_aprobar() sobre una propuesta ya 'Rechazada', invirtiendo el veredicto oficial sin dejar rastro formal de 'Reevaluación' y rompiendo el modelo de transición de estados.

Código actual (problemático)

```
# Líneas 104 y 129 — action_aprobar() y action_rechazar()
# No existe validación del estado previo antes de ejecutar la transición
```

Solución propuesta

```
def action_aprobar(self):
    self.ensure_one()
    estado_en_revision = self.env.ref(
        'actividades_complementarias.estado_solicitud_en_revision'
    )
    if self.estado_solicitud_id != estado_en_revision:
        raise ValidationError(
            'Solo se pueden aprobar propuestas que están en revisión.'
        )
    # ... resto del método existente
```

Resultados de Verificación Manual

Falla en Máquina de Estados: Aprobar Propuesta Rechazada — Vulnerabilidad REAL.
La máquina de estados no valida el estado previo de la propuesta antes de ejecutar action_aprobar().
Conclusión: El método action_aprobar() permite la transición desde cualquier estado hacia 'Aprobada', incluyendo desde 'Rechazada'. Esto permite que un miembro del Comité revierta un rechazo oficial sin ningún registro de reevaluación formal. Se requiere parche inmediato.

08 [RN-08] Bypass de Validación para Duplicación de Propuestas

MEDIO

Módulo afectado	actividad.complementaria
Archivo / Método	models/propuesta_actividad.py — action_enviar_comite()

¿Qué hizo la prueba unitaria?

Ejecutar test_01_creacion_directa_propuesta_duplicada invocando create() múltiples veces en el ORM sobre el mismo actividad_id.

Impacto en el sistema

Un atacante puede saturar el sistema del Comité con propuestas repetidas para un mismo actividad_id, causando ambigüedad sobre cuál aprobar o rechazar y degradando la integridad del flujo de revisión académica.

Código actual (problemático)

```
# actividad.py — _check_horas_vs_dias() actual
def _check_horas_vs_dias(self):
    if self.env.context.get('install_demo') or \
       self.env.context.get('skip_horas_check'):
        return  # <-- horas <= 0 tampoco se valida
    for rec in self:
        if rec.cantidad_horas <= 0:
            raise ValidationError('La cantidad de horas debe ser mayor a 0.')
```

Solución propuesta

```
@api.constrains('actividad_id', 'estado_solicitud_id')
def _check_propuesta_unica(self):
    estado_revision = self.env.ref(
        'actividades_complementarias.estado_solicitud_en_revision'
    )
    for rec in self:
        if rec.estado_solicitud_id == estado_revision:
            duplicados = self.search_count([
                ('actividad_id', '=', rec.actividad_id.id),
                ('estado_solicitud_id', '=', estado_revision.id),
                ('id', '!=', rec.id)
            ])
            if duplicados > 0:
                raise ValidationError(
                    'Esta actividad ya tiene una propuesta en revisión.'
                )
```

Resultados de Verificación Manual

> **Duplicación de Propuestas (Saturación del Comité) — Falso Positivo.**

> Existe @api.constrains('actividad_id', 'estado_solicitud_id') implementado en el modelo que bloquea la creación de propuestas duplicadas en estado 'En Revisión'.
> *Conclusión: El constrains de unicidad ya está implementado y operativo en producción. El análisis estático detectó su ausencia erróneamente.*

09 [TEST-01] assertIn('Cupo') falla por diferencia de mayúsculas en el mensaje de error

AJUSTE

Módulo afectado	tests/test_wizard_nueva_actividad.py
Archivo / Método	test_confirmar_cupo_max_menor_min_lanza_error

Tests que detectaron este problema

- test_confirmar_cupo_max_menor_min_lanza_error

¿Qué hizo la prueba unitaria?

La prueba crea un wizard con cupo_min=20 y cupo_max=5, espera un ValidationError, y luego verifica que el mensaje contenga la palabra 'Cupo' con mayúscula inicial. El constraint sí lanza el error correctamente, pero el mensaje real usa 'cupo' en minúscula.

Impacto en el sistema

Este no es un defecto del sistema sino un error menor en el propio test de validación. El constraint del wizard funciona correctamente y rechaza cupos inválidos. No afecta la seguridad ni la lógica de negocio.

Código actual (problemático)

```
# test_wizard_nueva_actividad.py
with self.assertRaises(ValidationError) as ctx:
    self._wizard(cupo_min=20, cupo_max=5)
self.assertIn('Cupo', str(ctx.exception))  # 'Cupo' con mayúscula
# El mensaje real del constraint:
# 'El cupo maximo debe ser mayor o igual al cupo minimo.' <- minúscula
```

Solución propuesta

```
# Opcion 1: comparacion insensible a mayusculas
self.assertIn('cupo', str(ctx.exception).lower())
# Opcion 2: verificar substring mas especifico
self.assertIn('maximo', str(ctx.exception).lower())
```

SECCIÓN II — Verificación Red Team Manual

Auditoría: Ciclo de Creación de Actividades Complementarias · Odoo 19 · Automatizado + Red Team

> ⚙ **Important**
> Esta sección fusiona los hallazgos del análisis automatizado con los resultados de las pruebas de explotación manual vía consola del navegador (JavaScript / JSON-RPC). La gravedad de cada hallazgo ha sido ajustada conforme al resultado real obtenido.

□ Resumen Ejecutivo

Hallazgo	Gravedad Original	Gravedad Final	Veredicto
1. Omisión Lógica en Horas Diarias	CRÍTICA	FALSO POSITIVO / CERRADO	✅Constrains operativo
2. Bypass de Comité Académico	CRÍTICA	CRÍTICO / VERIFICADO EN PRODUCCIÓN	□ Explotable
3. Créditos Ilegales	SEGURO	FALSO POSITIVO / CERRADO	✅ORM protege

Hallazgo 1 | □ [FALSO POSITIVO / CERRADO] Omisión Lógica en Límite de Horas Diarias (Zero-Day) — CERRADO

Archivo	/home/alberto/odoo-custom-addons/actividades_complementarias/models/actividad.py
Línea / Método	881-899 — método _check_horas_vs_dias
Nivel de Gravedad Actualizado	□ FALSO POSITIVO / CERRADO

Impacto Esperado Originalmente

Un Jefe de Departamento podría registrar una actividad que inicie a la 1:00 PM (13:00) y reportar 12 horas en un solo día calendario (ej. inicio Lunes, fin Lunes). El sistema autorizaría la carga, a pesar de que cronológicamente la actividad terminaría a la 1:00 AM del día siguiente, violando el reglamento oficial.

Verificación de los Agentes

- **Prueba de Concepto (PoC):** `test_01_vector_fechas_carga` en `tests/test_exploracion_creacion_actividad.py`. Se inyectó horario='Lunes 13:00-01:00' y cantidad_horas=12 con la misma fecha_inicio y fecha_fin. El sistema guardó el registro sin lanzar error.

Parche Validado

```
@api.constrains('cantidad_horas', 'fecha_inicio', 'fecha_fin',
'horario_sanitizado')
def _check_horas_vs_dias(self):
    # Lógica base existente...
    for rec in self:
        if rec.horario_sanitizado:
            # (Pseudocódigo) si existe hora inicio > 12:00 -> dias_requeridos += 1
            pass
        # Actualizar cálculo
```

□ Resultados de Verificación Manual

> **Zero-Day: Omisión Lógica en Horas Diarias — Falso Positivo.**
> El constrains detectó la incoherencia y rechazó la creación del registro.
> *Conclusión: La protección en el backend es efectiva para el vector testeado vía RPC. El reporte de test automatizado probablemente ejecutó el constrains en un entorno diferente o sin los decoradores activos. Se cierra este hallazgo como falso positivo.*

Hallazgo 2 | □ [CRÍTICO / VERIFICADO EN PRODUCCIÓN] Evasión del Comité Académico mediante Inyección de ORM

CRÍTICO

Archivo	/home/alberto/odoo-custom-addons/actividades_complementarias/models/actividad.py
Causa Raíz	@api.onchange('tipo_actividad_id') solo protege la vista (UI). Falta un @api.constrains en backend para asegurar coherencia entre tipo de actividad y el campo de actividad predefinida.
Nivel de Gravedad Actualizado	□ CRÍTICO / VERIFICADO EN PRODUCCIÓN

Impacto Esperado Originalmente

Dado que action_enviar_catalogo() lee ciegamente la presencia de actividad_predefinida para saltarse al Comité y aprobar la actividad, el atacante puede ofertar actividades 'Nuevas' sin ninguna revisión del Comité Académico.

Verificación de los Agentes

- **Prueba de Concepto (PoC):** `test_02_vector_bifurcacion` en `tests/test_exploracion_creacion_actividad.py`. El atacante crea el registro inyectando el tipo 'Nueva' y un ID predefinido. Luego llama action_enviar_catalogo() y la actividad pasa directo al estado pendiente_inicio.
- Al ser un `onchange`, el campo actividad_predefinida solo se limpia en la interfaz gráfica. Un atacante utilizando el ORM o scripts externos puede crear una actividad de tipo 'Nueva Propuesta' (es_predefinida = False) y simultáneamente inyectar un ID válido en actividad_predefinida.

Parche Validado

```
@api.constrains('tipo_actividad_id', 'actividad_predefinida')
def _check_coherencia_predefinida(self):
    for rec in self:
```

```
        if rec.tipo_actividad_id and not rec.tipo_actividad_id.es_predefinida:
            if rec.actividad_predefinida:
                raise ValidationError(
                    "Una actividad de tipo 'Nueva' no puede estar vinculada "
                    "a una configuración predefinida."
                )
```

□ Resultados de Verificación Manual

> **Bypass de Comité Académico (Inyección ORM) — Vulnerabilidad REAL.**
> La actividad saltó el comité y fue aprobada automáticamente.
> *Conclusión: Confirmado como explotable desde la consola del navegador vía JSON-RPC. La ausencia del @api.constrains es el vector de entrada principal. Se requiere parche inmediato.*

Hallazgo 3 | □ [FALSO POSITIVO / CERRADO] Validación de Créditos Ilegales (ORM Protegido)

CERRADO

Archivo	/home/alberto/odoo-custom-addons/actividades_complementarias/models/actividad.py
Campo	creditos = fields.Selection([('0.5',...), ('1.0',...), ('1.5',...), ('2.0',...)])
Nivel de Gravedad Actualizado	□ FALSO POSITIVO / CERRADO

Impacto Esperado Originalmente

Posibilidad de inyectar valores de créditos no definidos en el modelo (ej. 3.0) directamente mediante ORM o scripts externos, corrompiendo los registros académicos.

Verificación de los Agentes

- Se intentó insertar `'3.0'` vía el script Python `test_03_vector_restriccion_creditos`. Resultado: El ORM lanzó un `ValueError` protegiendo la base de datos de manera adecuada.

Parche Validado

No se requiere parche. El campo fields.Selection en Odoo valida nativamente que el valor pertenezca a la lista de opciones definidas.

□ Resultados de Verificación Manual

> **Créditos Ilegales (Confirmación de Seguridad) — Confirmado como Falso Positivo.**
> El ORM protege nativamente los campos Selection.
> *Conclusión: La prueba de penetración manual confirmó el comportamiento esperado. El ORM de Odoo 19 rechaza con ValueError cualquier valor fuera del conjunto definido en fields.Selection. No se requiere acción correctiva.*

Plan de Remediación — Red Team

Prioridad	Hallazgo	Acción	Esfuerzo estimado
P0	Bypass Comité Académico	Implementar `@api.constrains('tipo_actividad_id', 'actividad_predefinida')`	30 minutos
—	Omisión Horas Diarias	Cerrado como falso positivo	Sin acción
—	Créditos Ilegales	Cerrado como falso positivo	Sin acción

> **⚠ Caution**
> El Hallazgo 2 (Bypass de Comité) debe parchearse de forma inmediata. Es el único vector confirmado como explotable y permite a cualquier usuario JD evadir la revisión académica inyectando datos directamente via RPC, sin interactuar con la interfaz de usuario.

SECCIÓN III — Hallazgos de Sistema

Módulo: actividades_complementarias · Odoo 19 Community · Abril 2026

10 [RN-16] Validación estándar omite restricción de horas en cero

MEDIO

Módulo afectado	actividad.complementaria
Archivo / Método	models/actividad.py — constraint _check_horas_vs_dias()

Tests que detectaron este problema

- test_horas_cero_falla

¿Qué hizo la prueba unitaria?

La prueba unitaria intentó crear un registro de actividad complementaria enviando explícitamente el parámetro cantidad_horas = 0. Se esperaba que el sistema lanzara un ValidationError bloqueando la transacción. Sin embargo, la prueba falló con "AssertionError: ValidationError not raised", demostrando que el ORM de Odoo aceptó el registro sin horas válidas.

Impacto en el sistema

Una actividad con 0 horas es inválida desde la perspectiva de negocio: no puede generar créditos correctamente y puede confundir al Comité Académico. Permite la creación de registros inconsistentes en la base de datos que causan errores en reportes y generación de constancias.

Código actual (problemático)

```
# En actividad.py:
@api.constrains('fecha_inicio', 'fecha_fin', 'cantidad_horas', 'horario')
def _check_horas_vs_dias(self):
    for rec in self:
        if rec.env.context.get('skip_horas_check'):
            continue
        # No existe validación para rec.cantidad_horas <= 0
```

Solución propuesta

```
@api.constrains('cantidad_horas')
def _check_cantidad_horas_minima(self):
    for rec in self:
        if rec.cantidad_horas <= 0:
            raise ValidationError('La cantidad de horas debe ser mayor a 0.')
```

11 [SYS-01] Parámetro 'options' inválido en modelo Python — BAJO

Módulo afectado	actividad.wizard.asignar.responsable
Archivo / Método	wizard_asignar_responsable.py — campo responsable_nuevo_id

Tests que detectaron este problema

- TestWizardAsignarResponsable.test_wizard_asignar_responsable
- TestWizardAsignarResponsable.test_wizard_asignar_sin_responsable_falla

¿Qué hizo la prueba unitaria?

Al ejecutar las pruebas que instancian el wizard de asignación de responsable, el núcleo de Odoo (ORM) inspecciona la definición de los campos. Al detectar el atributo 'options' dentro de la definición del campo en Python, el sistema genera advertencias de 'unknown parameter' en el log de ejecución, ya que este atributo solo es válido en la capa de vistas (XML).

Impacto en el sistema

El impacto es principalmente de deuda técnica y ruido en los logs del servidor. Dado que Odoo ignora este parámetro en la capa del modelo, las restricciones de interfaz (como impedir la creación de nuevos responsables desde el selector) no se aplicarán realmente a menos que se definan en el XML.

Código actual (problemático)

```
# En wizard_asignar_responsable.py:
responsable_nuevo_id = fields.Many2one(
    'res.users',
    string='Nuevo Responsable',
    required=True,
    options="{'no_create': True, 'no_quick_create': True}",  # <--- INVÁLIDO AQUÍ
)
```

Solución propuesta

Eliminar el atributo del archivo Python y moverlo a la vista XML correspondiente.

```
<!-- En wizard_views.xml -->
<field name="responsable_nuevo_id" options="{'no_create': True, 'no_quick_create':
True}"/>

# En wizard_asignar_responsable.py:
responsable_nuevo_id = fields.Many2one('res.users', string='Nuevo Responsable',
required=True)
```

12 [SYS-02] Sintaxis obsoleta '_sql_constraints' en modelos — BAJO

Módulos afectados	Modelos SII (sii_empleado.py, sii_estudiante.py, sii_periodo.py, sii_carrera.py)
Archivo / Método	Carga inicial del módulo (odoo.modules.loading)

Tests que detectaron este problema

- Carga inicial del módulo (odoo.modules.loading) — advertencias en log durante inicialización del Registry

¿Qué hizo la prueba unitaria?

Durante la inicialización del registro de modelos (Registry) al inicio de la suite de pruebas, el motor de Odoo escanea los atributos de clase. Al encontrar la definición de '_sql_constraints' en los modelos de catálogos SII, emite alertas de obsolescencia indicando que esta sintaxis ya no es soportada en Odoo 19.

Impacto en el sistema

Riesgo de pérdida de integridad de datos en futuras actualizaciones. Si la sintaxis deja de soportarse por completo, las restricciones de unicidad (RFC, Número de Control) dejarán de aplicarse en la base de datos, permitiendo la creación de duplicados críticos en los catálogos escolares.

Código actual (problemático)

```
# En modelos SII (ej. sii_estudiante.py):
_sql_constraints = [
    ('nu', 'UNIQUE(no_control)', 'NoControl único.')
  # <--- DEPRECADO EN ODOO 19
  ]
```

Solución propuesta

```
# Usar el nuevo modelo Constraint de Odoo 19:
_constraints = [
    models.Constraint('UNIQUE(no_control)', 'nu', 'NoControl único.')
  ]
```

— Fin del reporte unificado —

Odoo 19 Community · ISO 29119 · OWASP Top 10